ARIA — Autonomous Research & Intelligence Agent

7-Day Spike Curriculum

> **Philosophy:** Build first, understand after. Each day has a clear goal, a set of tasks, and an expected output. Do not look for tutorials — figure it out. When you're stuck for more than 30 minutes, bring your attempt to the discussion, not a blank slate.

Project Overview

What you are building:
A production-grade agentic research backend that takes a topic, autonomously plans a research strategy, orchestrates multiple tools, and returns a structured cited intelligence report.

Tech Stack:

- LLM: Qwen3 via Ollama (local)
- Agent Framework: LangGraph
- Backend: FastAPI
- Task Queue: Celery + Redis
- Database: PostgreSQL + SQLAlchemy (async)
- Observability: Langfuse
- Containerization: Docker Compose
- Config: pydantic-settings
- Linting/Typing: ruff + mypy

Project Structure (build toward this):

```
aria/
├── api/
│   ├── routes/
│   ├── middleware/
│   └── dependencies/
├── agent/
│   ├── graph.py
```

```
│   ├── state.py
│   └── nodes/
│       ├── planner.py
│       ├── router.py
│       ├── evaluator.py
│       ├── synthesizer.py
│       └── critic.py
├── tools/
│   ├── search/
│   │   ├── base.py
│   │   ├── mock.py
│   │   └── arxiv.py
│   ├── retrieval/
│   │   ├── base.py
│   │   └── mock.py
│   └── extraction/
│       ├── base.py
│       └── mock.py
├── llm/
│   ├── base.py
│   ├── ollama.py
│   └── mock.py
├── workers/
│   └── research_task.py
├── db/
│   ├── models/
│   ├── migrations/
│   └── repositories/
├── evaluation/
│   ├── judges/
│   └── metrics/
├── core/
│   ├── config.py
│   ├── logging.py
│   └── exceptions.py
├── tests/
├── docker-compose.yml
├── pyproject.toml
└── README.md
```

Day 1 — Tool Use from Scratch

Goal

Understand what tool use is at the protocol level before any framework touches it. By end of day you must be able to explain: what a tool definition is, how the LLM signals it wants to call a tool, and how you send the result back.

What to build

A raw tool-calling loop in plain Python. No LangChain. No LangGraph. Just Ollama + Python.

The program takes a hardcoded research question, gives the LLM two tools to choose from, executes whichever tool the LLM picks, sends the result back, and prints the final answer.

Tasks

1. Set up the project with `uv`. Install: `ollama`, `python-dotenv`, `rich`.
2. Create `llm/base.py` — an abstract base class `BaseLLM` with at minimum:
    - a `complete()` method
    - a `complete_with_tools()` method
      Think carefully about what parameters each method needs.
3. Create `llm/ollama.py` — `OllamaLLM` that implements `BaseLLM` using the Ollama Python client.
4. Create `tools/search/base.py` — an abstract `BaseSearchTool` with a `search()` method that returns a list of `SearchResult` objects. Define `SearchResult` as a dataclass or Pydantic model.
5. Create `tools/search/mock.py` — `MockSearchTool` that returns hardcoded fake results. It must follow `BaseSearchTool`.
6. Create `core/config.py` — using `pydantic-settings`, load at minimum:
   `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `ENV`.
7. Create `main.py` — the raw tool-calling loop:
    - Define the tool schema (what the LLM needs to know about each tool)
    - Send question + tool definitions to the LLM

- Parse the LLM response to detect if it wants to call a tool
- Execute the tool
- Send result back to LLM
- Print final answer with `rich`

Expected Output

When you run `python main.py` the terminal should show something like:

```
╭─ ARIA Research Agent ──────────────────────────────╮
│  Question: What are the latest advances in LLMs?   │
╰────────────────────────────────────────────────────╯

[LLM DECISION]  → wants to call: search
                  with query: "latest LLM advances 2024"

[TOOL EXECUTED] → MockSearchTool returned 3 results
                  Result 1: "Attention is All You Need..."
                  Result 2: "GPT-4 Technical Report..."
                  Result 3: "LLaMA 2 Paper..."

[LLM FINAL]     → Based on the search results, the latest
                  advances in LLMs include...
```

Key Questions to Answer Before Moving to Day 2

Before moving on, make sure you can answer these without looking at your code:

- What is the exact format the Ollama API expects for tool definitions?
- What does the LLM response look like when it decides to call a tool vs when it answers directly?
- Who runs the loop — the LLM or your code?
- What happens if the LLM tries to call a tool you didn't define?

Day 2 — The ReAct Loop in Pure Python

Goal

Build the full agent loop manually. The agent must be able to call tools multiple times, reason about results, and decide when it has enough information to stop. Feel the pain of managing state manually — this pain is exactly what LangGraph solves on Day 3.

What to build

A multi-turn agent loop that:

- Takes a research question
- Calls tools multiple times if needed
- Tracks the full history of thoughts, actions, and observations
- Decides when to stop and synthesize
- Produces a simple structured output

Tasks

1. Add a second mock tool — `tools/retrieval/mock.py` — `MockRAGTool` that simulates querying a knowledge base. Returns different fake results than the search tool.
2. Create `agent/state.py` — define an `AgentState` dataclass that holds:
    - The original question
    - List of research questions generated
    - History of tool calls and their results
    - Current iteration count
    - Final answer (optional, set when done)
      Think carefully about what else the agent needs to track.
3. Build the ReAct loop in `main.py` (or a new `agent/loop.py`):
    - **Reason:** LLM looks at question + history, decides what to do next
    - **Act:** Execute the chosen tool
    - **Observe:** Add result to history
    - **Repeat** until LLM says it is done OR iteration limit hit
    - Hard cutoff at 10 iterations
4. Add a simple **Planner step** before the loop:
    - LLM receives the topic
    - Outputs 3 research sub-questions
    - Loop runs for each sub-question
5. Print the full agent trace to terminal using `rich` — every thought, every tool call, every observation, clearly formatted.

Expected Output

```
╭─ ARIA Research Agent ──────────────────────────────────╮
│  Topic: Impact of LLMs on Healthcare                   │
╰────────────────────────────────────────────────────────╯

[PLANNER]
  → Sub-question 1: What LLM applications exist in diagnosis?
  → Sub-question 2: What are the safety risks?
  → Sub-question 3: What regulations govern AI in healthcare?

═══════════════════════════════════════════════════════

[ITERATION 1] Sub-question 1

═══════════════════════════════════════════════════════

  [THOUGHT]   I need to find current LLM applications in diagnosis.
              Web search is the right tool here.

  [ACTION]    search("LLM clinical diagnosis applications")

  [OBSERVE]   3 results returned:
              - "LLMs in Radiology: A Survey..."
              - "GPT-4 Diagnostic Accuracy Study..."
              - "AI Diagnosis Review 2024..."

  [THOUGHT]   Results look relevant. Moving to sub-question 2.

═══════════════════════════════════════════════════════

[ITERATION 2] Sub-question 2
...

[FINAL ANSWER]
  Research complete after 3 iterations.
  Sources evaluated: 9
  {
    "summary": "LLMs are being applied in...",
    "key_findings": [...],
    "sources": [...]
  }
```

Key Questions to Answer Before Moving to Day 3

- What problems did you hit managing state manually?

- What happens when the LLM produces inconsistent output format?
- How did you handle the iteration hard cutoff?
- What would happen if two sub-questions need to run in parallel?

Day 3 — Migrate to LangGraph

Goal

Understand why LangGraph exists by rebuilding your Day 2 loop as a proper graph. Add conditional routing — the agent can take different paths depending on what it finds.

What to build

The full ARIA agent graph with all five nodes: Planner → Router → Tools → Evaluator → Synthesizer → Critic. With conditional edges so the graph can loop back when results are insufficient.

Tasks

1. Add `langgraph` and `langchain-community` to your dependencies.
2. Redefine `agent/state.py` as a LangGraph `TypedDict` state schema. Every node reads from and writes to this shared state.
3. Create each node as a separate file in `agent/nodes/` :

   `planner.py` — receives topic, outputs list of research questions

   `router.py` — receives one research question, decides: search, RAG, or both. Returns tool selection.

   `evaluator.py` — receives tool results, scores them: are they relevant and sufficient? Returns: `continue` (need more research) or `done` (enough information).

   `synthesizer.py` — receives all gathered results, writes structured report.

   `critic.py` — reviews the report, identifies gaps. Returns: `approve` or `revise` with specific gap description.
4. Create `agent/graph.py` — wire everything together:
   - Define nodes
   - Define edges

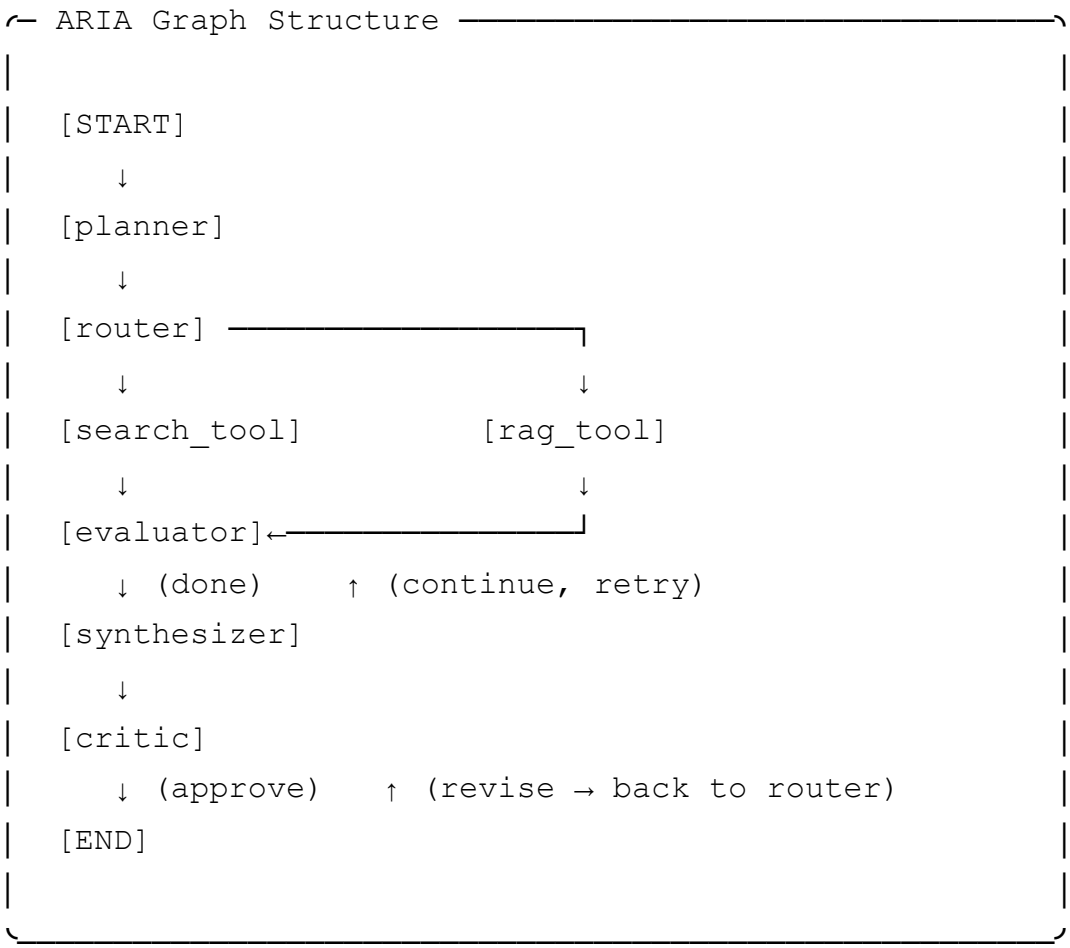
- Define **conditional edges** from Evaluator (loop back or continue) and Critic (approve or revise)
- Set entry point and finish point
- Compile the graph

5. Add a hard limit on loops — if Evaluator loops more than 3 times for one question, force continue.

6. Update `main.py` to run the LangGraph graph instead of your manual loop.

Expected Output

Same terminal output as Day 2 but now driven by the graph. Additionally, print the graph structure at startup:



Key Questions to Answer Before Moving to Day 4

- What does LangGraph give you that your manual loop didn't have?
- What is a conditional edge and how do you define one?
- What is the state schema and why does every node share it?
- How do you prevent infinite loops in a graph?

Day 4 — FastAPI + Celery + Redis

Goal

Wrap the agent in a production API. Agent runs become background tasks. The API is non-blocking — it returns immediately with a job ID and the client polls for results.

What to build

A FastAPI backend where:

- `POST /research` creates a job and returns a job ID immediately
- The agent runs inside a Celery worker in the background
- `GET /research/{job_id}` returns current status and results when done
- `GET /research/{job_id}/trace` returns the full agent trace

Tasks

1. Add dependencies: `fastapi`, `celery`, `redis`, `sqlalchemy`, `asyncpg`, `alembic`, `uvicorn`.
2. Create `core/config.py` — extend with: `DATABASE_URL`, `REDIS_URL`, `CELERY_BROKER_URL`.
3. Create `db/models/` — define at minimum:
    - `Job` model: id, tenant_id, status, topic, created_at, updated_at
    - `JobResult` model: job_id, report (JSON), sources, confidence_score
    - `AgentTrace` model: job_id, node_name, input, output, timestamp
4. Run Alembic migrations.
5. Create `db/repositories/` — one repository class per model. No raw SQL in routes.
6. Create `workers/research_task.py` — a Celery task that:
    - Receives job_id and topic
    - Runs the LangGraph agent
    - Updates job status in DB throughout (pending → running → complete/failed)
    - Saves result and trace to DB
7. Create `api/routes/research.py` — three endpoints:

- `POST /research` — validate input, create job in DB, enqueue Celery task, return job_id
- `GET /research/{job_id}` — fetch job status and result from DB
- `GET /research/{job_id}/trace` — fetch full agent trace

8. Create `docker-compose.yml` with services: `api`, `worker`, `redis`, `postgres`.

Expected Output

```
# Terminal 1 — start everything
docker-compose up

# Terminal 2 — submit a research job
curl -X POST http://localhost:8000/research \
  -H "Content-Type: application/json" \
  -d '{"topic": "Impact of LLMs on healthcare"}'

# Response — immediate, non-blocking
{
  "job_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "pending",
  "created_at": "2024-01-15T10:30:00Z"
}

# Terminal 2 — poll for status
curl http://localhost:8000/research/550e8400-e29b-41d4-a716-446655440000

# Response while running
{
  "job_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "running",
  "current_node": "evaluator",
  "iterations": 3
}

# Response when done
{
  "job_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "complete",
  "result": {
    "title": "Impact of LLMs on Healthcare",
    "summary": "...",
    "sections": [...],
    "citations": [...],
```

```
    "confidence_score": 0.87,
    "sources_evaluated": 12
  }
}
```

Key Questions to Answer Before Moving to Day 5

- Why does the API return immediately instead of waiting for the agent?
- What is Redis doing in this architecture?
- What is the difference between a Celery task and an async FastAPI route?
- What happens to a running job if the worker crashes?

Day 5 — Multi-tenancy + Resilience

Goal

Make the system safe for multiple tenants and robust against failures. No tenant should see another tenant's data. No single tool failure should crash an agent run.

What to build

Tenant-scoped data isolation and a resilience layer with retry logic and fallback tools.

Tasks

1. Add `Tenant` model to DB: id, name, api_key (hashed), created_at.
2. Create `api/middleware/auth.py` — API key authentication middleware:
    - Extract API key from `Authorization` header
    - Look up tenant in DB
    - Attach tenant to request state
    - Reject unknown keys with 401
3. Update all DB queries — every query must filter by `tenant_id`. A tenant must never be able to access another tenant's jobs or results. Enforce this at the repository layer, not the route layer.
4. Create `core/exceptions.py` — define custom exceptions:
    - `ToolExecutionError`

- `LLMError`
- `RateLimitError`
- `TenantNotFoundError`

5. Build a resilience wrapper for tools — for each tool call:
   - Retry up to 3 times with exponential backoff on failure
   - If all retries fail, try the fallback tool
   - If fallback also fails, return a `ToolFailureResult` with error info
   - Agent continues with partial results rather than crashing
6. Build the fallback chain:

```
ArxivTool fails → MockTool (in dev) / NewsAPITool (in prod)
RAGTool fails   → return empty results with warning flag
```

7. Add tenant context to all Celery tasks — every task knows which tenant it belongs to.

Expected Output

```
# Two different tenants, isolated data

# Tenant A submits job
curl -X POST http://localhost:8000/research \
  -H "Authorization: Bearer tenant_a_api_key" \
  -d '{"topic": "quantum computing"}'
# Returns job_id_A

# Tenant B submits job
curl -X POST http://localhost:8000/research \
  -H "Authorization: Bearer tenant_b_api_key" \
  -d '{"topic": "climate change"}'
# Returns job_id_B

# Tenant A tries to access Tenant B's job → 403
curl http://localhost:8000/research/job_id_B \
  -H "Authorization: Bearer tenant_a_api_key"
# {"error": "forbidden", "detail": "job not found"}

# Tool failure handled gracefully
# Worker logs show:
[RESILIENCE] ArxivTool failed (attempt 1/3): ConnectionTimeout
[RESILIENCE] ArxivTool failed (attempt 2/3): ConnectionTimeout
[RESILIENCE] ArxivTool failed (attempt 3/3): ConnectionTimeout
```

```
[RESILIENCE] Falling back to MockSearchTool
[RESILIENCE] Fallback succeeded. Continuing with partial results.
```

Key Questions to Answer Before Moving to Day 6

- Where exactly do you enforce tenant isolation — route, service, or repository layer?
- What is exponential backoff and why is it better than fixed retry delay?
- What is the difference between a fallback and a retry?
- What should the agent do when ALL tools fail for a sub-question?

Day 6 — Observability + Evaluation

Goal

Make the system inspectable and measurable. Every decision the agent makes should be traceable. Every report it produces should be scored.

What to build

Full Langfuse tracing on every node and an LLM-as-judge evaluation framework that scores report quality.

Tasks

1. Add dependencies: `langfuse`.

2. Instrument every agent node with Langfuse tracing:

    - Each node creates a Langfuse span
    - Span includes: node name, input state, output state, duration, token usage
    - Each LLM call inside a node creates a child span
    - Each tool call creates a child span

3. Create `evaluation/judges/` — an LLM-as-judge that scores a completed report on:

    - **Faithfulness** (0-1): Are all claims supported by the cited sources?
    - **Relevance** (0-1): Does the report actually answer the original topic?
    - **Coverage** (0-1): Are the research sub-questions all addressed?
    - **Confidence** (0-1): Overall quality score

The judge is itself an LLM call with a carefully designed prompt and structured output.

4. Create `evaluation/metrics/` — deterministic metrics (no LLM needed):
    - Citation count
    - Source diversity (how many different sources)
    - Report length
    - Iteration count (fewer iterations = more efficient)
5. Run evaluation automatically after every agent run. Save scores to DB alongside the result.
6. Add structured logging throughout using `structlog` — every log line should be JSON with: timestamp, level, tenant_id, job_id, node, message.

Expected Output

```
# Langfuse dashboard shows full trace:
Research Job: 550e8400
├── planner (234ms)
│   └── llm_call: claude-3 (450 tokens)
├── router [question_1] (12ms)
├── search_tool [question_1] (891ms)
├── evaluator [question_1] (198ms)
│   └── llm_call: claude-3 (280 tokens)
│   └── decision: "insufficient, retry"
├── search_tool [question_1 retry] (743ms)
├── evaluator [question_1 retry] (201ms)
│   └── decision: "sufficient, continue"
├── synthesizer (567ms)
│   └── llm_call: claude-3 (1200 tokens)
└── critic (334ms)
    └── decision: "approve"

# Evaluation scores saved with result:
{
  "job_id": "550e8400",
  "scores": {
    "faithfulness": 0.92,
    "relevance": 0.88,
    "coverage": 0.95,
    "confidence": 0.91
  },
  "metrics": {
    "citation_count": 8,
```

```
    "source_diversity": 6,
    "iterations": 5,
    "total_tokens": 3847
  }
}
```

Key Questions to Answer Before Moving to Day 7

- What is the difference between observability and logging?
- What makes LLM-as-judge different from a deterministic metric?
- Why do you need both LLM-based and deterministic evaluation?
- How would you use these scores to improve the agent over time?

Day 7 — Polish + Demo Readiness

Goal

Make the project presentable, runnable in one command, and easy to understand for someone seeing it for the first time. This is your portfolio piece.

Tasks

1. `docker-compose.yml` — full stack in one file:
    - `postgres` — database
    - `redis` — message broker
    - `api` — FastAPI (with hot reload in dev)
    - `worker` — Celery worker
    - `flower` — Celery monitoring dashboard

      All services start with `docker-compose up`. No manual steps.
2. Alembic migrations run automatically on API startup.
3. A seed script that creates two test tenants with API keys.
4. `README.md` — must include:
    - What ARIA is (2-3 sentences)
    - Architecture diagram (ASCII is fine)
    - How to run it (exact commands, nothing assumed)

    - How to submit a research job (curl examples)
    - Tech decisions section — why LangGraph, why Celery, why this structure
    - Known limitations and what you'd do next
5. `pyproject.toml` — ruff + mypy fully configured, CI passing.
6. Clean up all TODOs, dead code, and debug print statements.
7. Record a demo run — terminal showing a full research job from submission to result.

Final Expected Output

```
# Clone and run — nothing else needed
git clone https://github.com/you/aria
cd aria
cp .env.example .env
docker-compose up

# In another terminal — run the seed script
python scripts/seed.py

# Submit a research job
curl -X POST http://localhost:8000/research \
  -H "Authorization: Bearer test_tenant_key" \
  -H "Content-Type: application/json" \
  -d '{"topic": "The future of agentic AI systems"}'

# Watch it run in Flower dashboard
open http://localhost:5555

# Check Langfuse traces
open http://localhost:3000

# Get the result
curl http://localhost:8000/research/{job_id} \
  -H "Authorization: Bearer test_tenant_key"
```

What You Can Say in Interviews After This

- "I built a multi-node agentic workflow in LangGraph with conditional routing, self-critique loops, and hard iteration limits"

- "The system uses async task execution via Celery so agent runs don't block the API — the client gets a job ID immediately and polls for results"
- "Every agent run is fully traced in Langfuse — I can replay any decision the agent made, see token usage per node, and identify bottlenecks"
- "I built an LLM-as-judge evaluation framework that scores every report on faithfulness, relevance, and coverage"
- "Multi-tenancy is enforced at the repository layer — tenant isolation is a guarantee, not a convention"
- "All external tools follow a base interface — swapping Arxiv for any other API is a one-file change"

General Rules for the Spike

1. **Build before asking.** If you are stuck, spend 30 minutes trying. Then bring your attempt, not a blank slate.
2. **No copy-paste from tutorials.** Read docs, understand, then write from your own understanding.
3. **Commit at the end of every day.** Your git history is part of the portfolio.
4. **When something does not work, read the error before asking.** What is it telling you?
5. **Mock aggressively in dev.** Do not waste time on API rate limits during learning.
6. **Type hint everything.** mypy should pass before you call a day done.